

Container Dashboard on FABRIC

Ethan Charest

CSC 478 Cloud Engineering
Project Deliverable 1
Fall 2026

Chapter 1: Vision

Every application in the cloud eventually has the same bad day: the machine it runs on goes away. A server reboots, a disk fills up, or traffic spikes past what one host can handle. What separates a fragile deployment from a resilient one is not whether that day comes, but what happens next.

In CSC 468 I built the Container Dashboard, a three-tier web app. It has an Nginx frontend, a Flask REST API, and a PostgreSQL database that stores a persistent log. It runs with Docker Compose on a single CloudLab virtual machine. That works, but everything depends on one host. If that VM goes down, the whole app goes down, and there's no way to spread the work across machines or scale up when needed.

For CSC 478, I want to keep the same application but move it onto a three-node Kubernetes cluster (RKE2) running on FABRIC. The whole environment, from the virtual machines to the running app, will be built by Jupyter notebooks. FABRIC slices are temporary and often need to be recreated, so automating every step makes the project reproducible. Kubernetes adds what a single Docker host can't: it restarts failed containers, moves work off a failed node, and scales parts of the app on demand.

Architecture

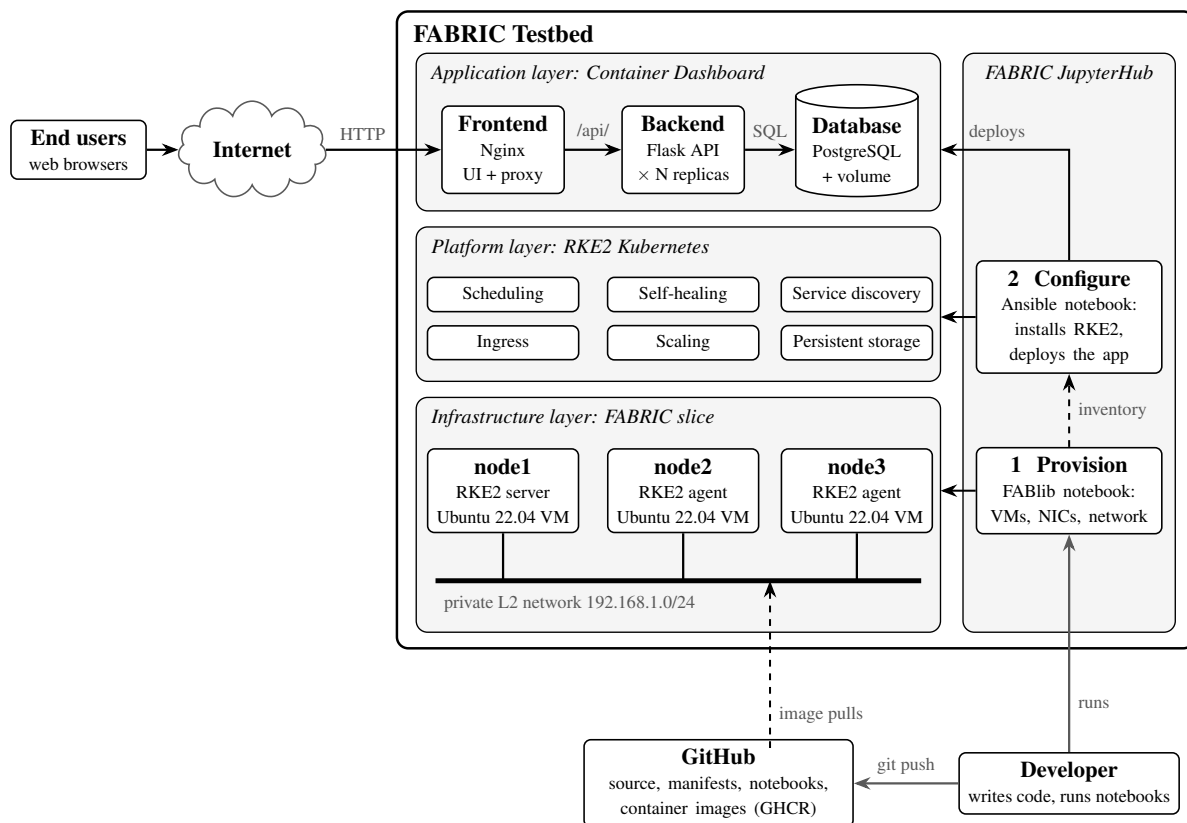


Figure 1. High-level architecture of the project.

The architecture is built in three layers, each created by a different tool and each depending only on the layer below it. At the bottom, the infrastructure layer is a FABRIC slice of three Ubuntu virtual machines connected by a private network, created by a FABlib notebook. In the middle, the platform layer is an RKE2 Kubernetes cluster installed by an Ansible notebook, with node1 acting as the server and node2 and

node3 joining as agents. This layer provides what the old setup was missing: scheduling across machines, automatic recovery, service discovery, and scaling. At the top, the application layer is the same frontend, backend, and database as before, now running as Kubernetes workloads instead of Docker Compose services.

Two other points are worth noting. First, users reach the application through a single entry point, the cluster's ingress, which forwards traffic to the frontend; the backend and database are never exposed outside the cluster. Second, nothing is built on the nodes by hand. The code, Kubernetes files, and notebooks live on GitHub, and the cluster pulls pre-built container images from GitHub's registry. Because of this, the entire system can be deleted and rebuilt at any FABRIC site with the same result.

Chapter 2: Proposed Implementation

The work will live in a new repository, separate from the CSC 468 project, and will be built in five steps. Each step produces something that can be tested on its own before moving to the next:

1. **Container images:** Build the frontend and backend images and push them to GitHub's container registry so the cluster can pull them.
2. **Provisioning notebook:** Use FABlib to find a FABRIC site with enough resources and create three VMs connected by a private network.
3. **Cluster notebook:** Use Ansible to install RKE2 on node1, join node2 and node3 to the cluster, and add storage for the database.
4. **Deploy the app:** Apply Kubernetes files that run the database with persistent storage, run the backend and frontend with multiple copies, and expose only the frontend to users. The services keep the same names the app already uses (frontend, backend, db), so the code barely needs to change.
5. **Test it:** Confirm the app can be rebuilt from scratch, recovers when a container or node fails, scales up the backend, and keeps its data when the database restarts.

A few design decisions shape this plan. The application code will change very little, because the original project already uses service names to connect its parts: Nginx forwards requests to backend, and Flask connects to a database host called db. By giving the Kubernetes services those same names, the existing code keeps working. Database credentials, which were written in plain text in the Docker Compose file, will move into a Kubernetes Secret. The RKE2 version will also be pinned so that every rebuild produces the same cluster, since RKE2 has recently changed defaults between releases, including which ingress controller it installs.

I also expect a few challenges. FABRIC sites do not always have free resources, so the provisioning notebook must be able to try other sites automatically. Some sites only offer IPv6 on their management network, while GitHub is reachable only over IPv4, so the nodes may need extra network configuration before they can download software and images. FABRIC slices also expire, which is another reason the whole build has to be automated. Finally, the simple storage option I plan to use keeps the database on a single node, so the database will run as one copy while the frontend and backend run as several.

By the end of the project, I expect to demonstrate a system that can be rebuilt from nothing by running two notebooks, keeps serving users when parts of it fail, and scales on demand. The dashboard itself will show this in action, since it already displays which container answered each request.

Ethan Charest

(484) 447-7600 | ethancharest1@gmail.com | www.linkedin.com/in/ethan-charess-99972530a | <https://github.com/ethancharest>

Professional Summary

Computer Science undergraduate with hands-on experience in cloud infrastructure, automation, and system optimization. Focused on expanding technical expertise in cloud and DevOps technologies while applying problem-solving skills to build efficient, scalable solutions.

Education

West Chester University, West Chester, PA

Bachelor of Science in Computer Science | 2024 – Present

Delaware County Community College, Downingtown, PA

Computer Science Transfer Program | 2022 – 2024

Experience

Infrastructure and Cloud Services Intern

Wawa, Inc., Media, PA | 2024

- Deployed and configured virtual cloud servers in AWS to support enterprise infrastructure.
- Automated patching and connectivity processes across multiple vCenters using scripting tools, improving Jenkins pipeline efficiency.
- Performed server patching and updates to resolve critical vulnerabilities and enhance system performance.
- Allocated system resources and implemented feature updates based on operational requirements.
- Collaborated with cross-functional IT teams to optimize non-production cloud environments, reducing costs and environmental impact.

Customer Sales Associate

Wawa, Inc., Parkesburg, PA & West Chester, PA | 2020 – Present

- Delivered high-quality customer service in fast-paced retail and food service environments.

- Trained and mentored new employees, contributing to a collaborative team culture.
 - Maintained compliance with food safety and operational standards.
-

Technical Skills

Programming Languages: Java, Python, JavaScript, Haskell, PowerShell, JSON, YAML

Cloud & DevOps: AWS, Azure, Docker, Jenkins, Ansible, Git, GitHub

Tools & Platforms: VMware, Citrix, Splunk, Datadog, Fabric

Frameworks & Systems: Jira, Confluence, ServiceNow, Workday